TECNOLÓGICO NACIONAL DE MÉXICO

Instituto Tecnológico de Aguascalientes

Materia: Programación Orientada a Objetos | Actividad: Simulación de Festival de Música

Estudiante: Joaquín García

Tema: Herencia, Polimorfismo y Constructores en Python

Repositorio de GitHub: https://github.com/joaquinrg132-png/festival-musica-poo

1. Objetivo y Fundamentación Teórica

El objetivo de esta práctica es diseñar e implementar un programa orientado a objetos que simula un festival musical interactivo, aplicando rigurosamente los dos pilares fundamentales de la POO solicitados:

- **Herencia:** Se definió una clase base `Artista` con los atributos comunes (`nombre`, `genero`, `popularidad`) y métodos esenciales (`presentarse`, `actuar`, `despedirse`). A partir de ella se derivaron las subclases especializadas `Cantante`, `DJ` y `Banda`, cada una utilizando la función `super().__init__(...)` para inicializar los atributos heredados y añadiendo sus propios atributos específicos (`cancion_mas_popular`, `estilo` e `integrantes`, respectivamente).
- **Polimorfismo:** Cada subclase sobreescribió el método `actuar()` con una implementación personalizada. En la función `iniciar_festival(lista_artistas)`, se recorre la colección de artistas y se invoca `actuar()` de manera uniforme sin evaluar condicionales de tipo (`isinstance` o comparaciones de clase), permitiendo que cada objeto responda según su propia forma de actuar.

2. Repositorio de GitHub

El código fuente del proyecto se encuentra disponible de forma pública en el siguiente enlace:
🌐 https://github.com/joaquinrg132-png/festival-musica-poo

3. Capturas de Pantalla de la Ejecución

A continuación se muestran las capturas de la terminal ejecutando el programa con los tres tipos de artistas (Cantante, DJ y Banda):

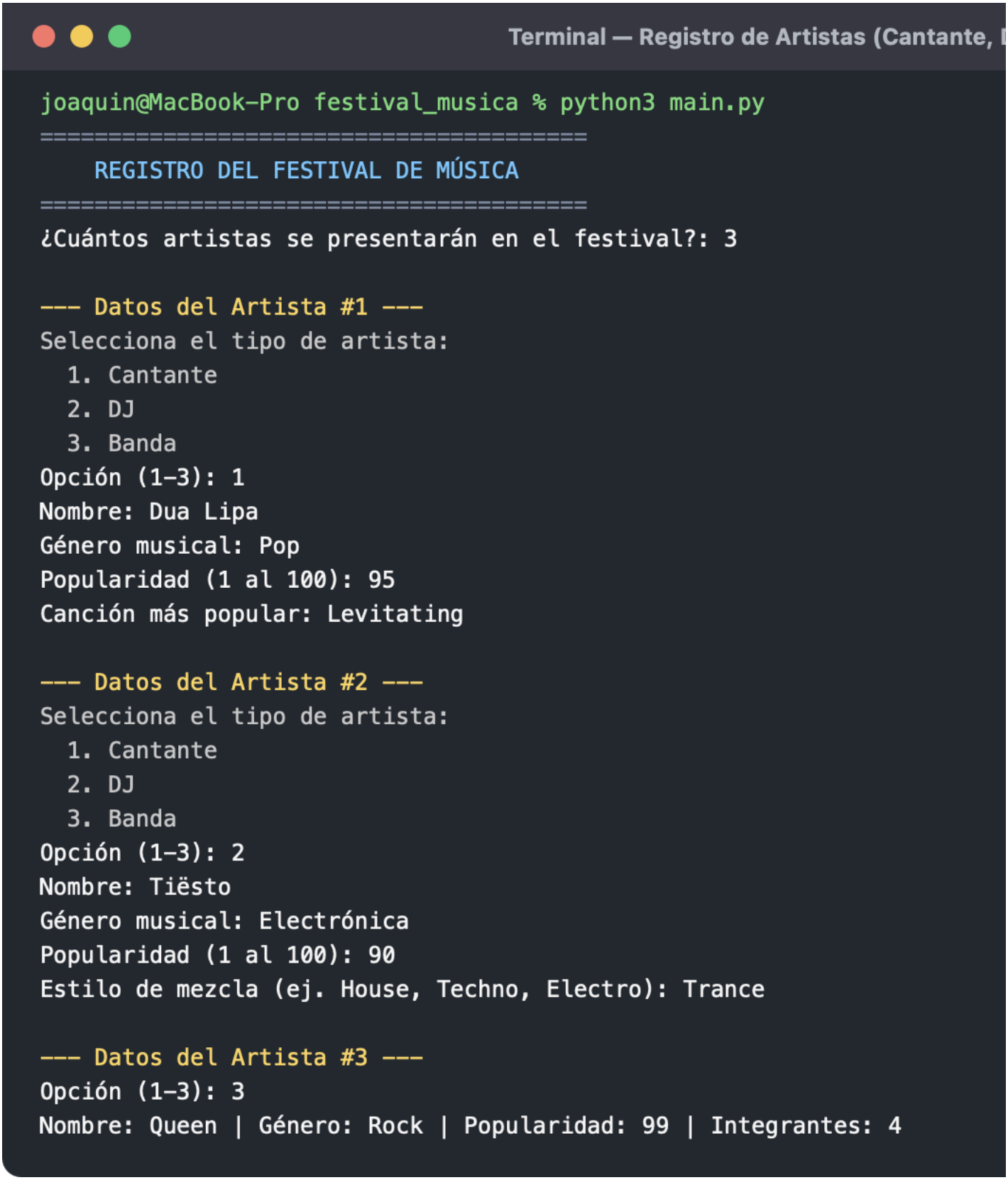


Figura 1: Registro interactivo de artistas solicitando atributos comunes y atributos específicos por subclase.

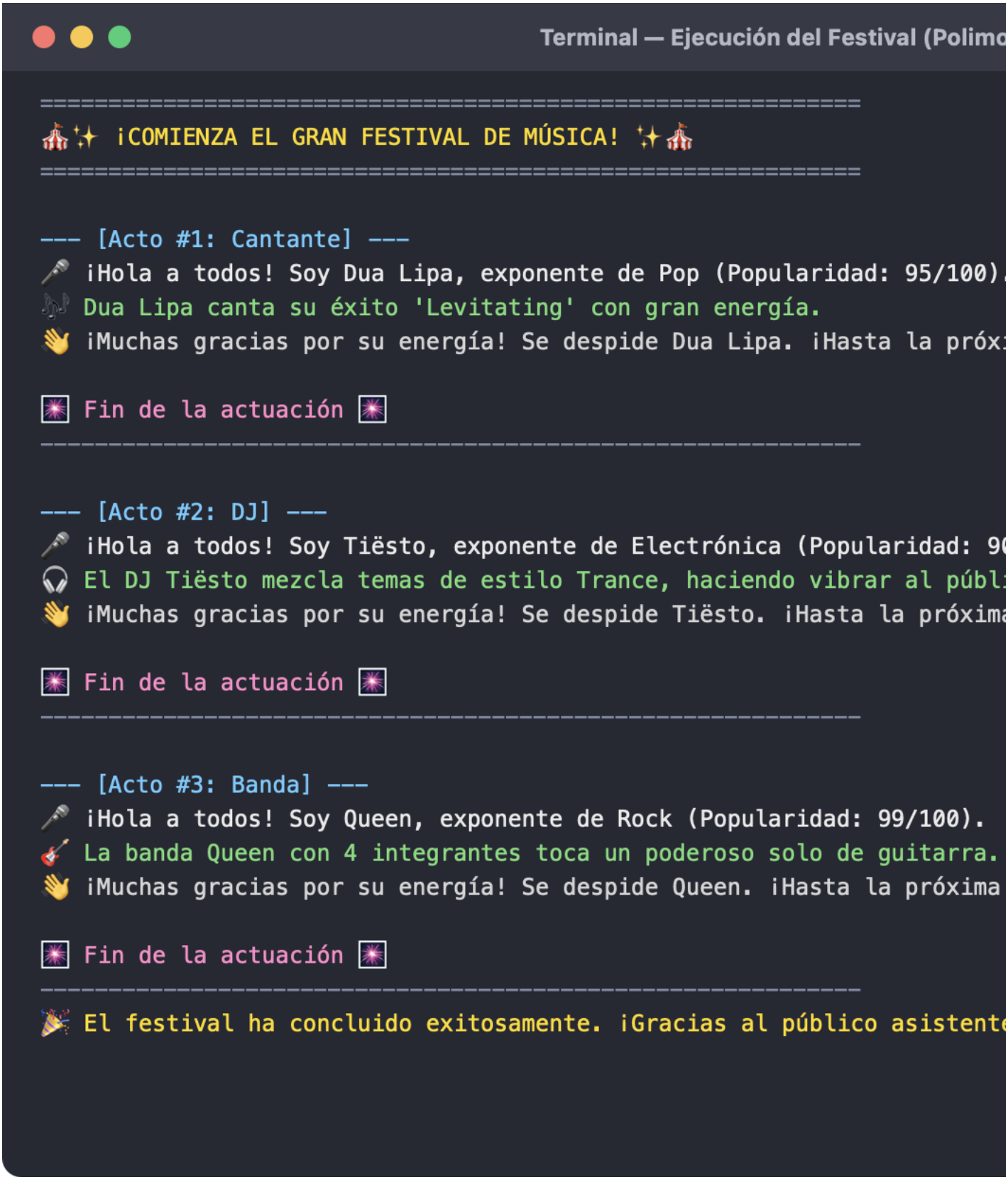


Figura 2: Ejecución del festival aplicando polimorfismo en actuar() y separación entre actos con "🎆 Fin de la actuación 🎆".

4. Código Fuente Completo (`main.py`)

Implementación en Python con validación de entradas, jerarquía de clases y

función polimórfica:

```
"""
Programa: Simulación de Festival de Música
Conceptos aplicados: Programación Orientada a Objetos (POO), Herencia y
Polimorfismo.
"""

# ==========================================
# 1️⃣ CLASE BASE: Artista
# ==========================================
class Artista:
    """Clase base que representa a un artista musical genérico."""

    def   init  (self, nombre: str, genero: str, popularidad: int):
        self.nombre = nombre
        self.genero = genero
        self.popularidad = popularidad

    def presentarse(self):
        """Presentación genérica del artista """
        print(f"🎤 ¡Hola a todos! Soy {self.nombre}, exponente de
{self.genero} (Popularidad: {self.popularidad}/100).")

    def actuar(self):
        """Método base que será sobrescrito por cada subclase
(polimorfismo) """
        print(f"🎵 {self.nombre} está realizando una presentación en el
escenario.")

    def despedirse(self):
        """Mensaje de despedida del artista """
        print(f"👋 ¡Muchas gracias por su energía! Se despide {self.nombre}.
¡Hasta la próxima!")


# ==========================================
# 2️⃣ SUBCLASES DE Artista (HERENCIA)
# ==========================================
class Cantante(Artista):
    """Subclase que representa a un cantante solista."""

    def   init  (self, nombre: str, genero: str, popularidad: int,
cancion mas popular: str):
        super().  init  (nombre, genero, popularidad)
        self.cancion_mas_popular = cancion_mas_popular

    def actuar(self):
        """Sobrescribe actuar() con el comportamiento específico de un
cantante """
        print(f"🎶 {self.nombre} canta su éxito '{self.cancion_mas_popular}'
con gran energía.")


class DJ(Artista):
    """Subclase que representa a un DJ."""

    def __init__(self, nombre: str, genero: str, popularidad: int, estilo:
str):
```

```
        super().__init__(nombre, genero, popularidad)
        self.estilo = estilo

    def actuar(self):
        """Sobrescribe actuar() con el comportamiento específico de un DJ """
        print(f"🎧 El DJ {self.nombre} mezcla temas de estilo {self.estilo},
haciendo vibrar al público.")


class Banda(Artista):
    """Subclase que representa a una banda musical."""

    def __init__(self, nombre: str, genero: str, popularidad: int,
integrantes: int):
        super().__init__(nombre, genero, popularidad)
        self.integrantes = integrantes

    def actuar(self):
        """Sobrescribe actuar() con el comportamiento específico de una
banda """
        print(f"🎸 La banda {self.nombre} con {self.integrantes} integrantes
toca un poderoso solo de guitarra.")


# ==========================================
# 3️⃣ FUNCIÓN DE INICIO DEL FESTIVAL (POLIMORFISMO)
# ==========================================
def iniciar_festival(lista_artistas):
    """
    Recorre la lista de artistas y ejecuta sus métodos.
    Aplica POLIMORFISMO: trata a todos los objetos como instancias de Artista
    sin necesidad de condicionales (if/elif) para verificar su tipo
específico.
    """
    if not lista_artistas:
        print("No hay artistas registrados para el festival.")
        return

    print("\n" + "=" * 60)
    print("🎪✨ ¡COMIENZA EL GRAN FESTIVAL DE MÚSICA! ✨🎪")
    print("=" * 60 + "\n")

    for i, artista in enumerate(lista_artistas, 1):
        print(f"--- [Acto #{i}] ---")
        artista.presentarse()
        artista.actuar()
        artista.despedirse()
        print("\n🎆 Fin de la actuación 🎆\n" + "-" * 60 + "\n")

    print("🎉 El festival ha concluido exitosamente. ¡Gracias al público
asistente! 🎉\n")


# ==========================================
# 4️⃣ PROGRAMA PRINCIPAL
# ==========================================
def pedir_entero_rango(mensaje: str, minimo: int, maximo: int) -> int:
    """Función auxiliar para validar números enteros dentro de un rango."""
    while True:
```

```
        try:
            valor = int(input(mensaje))
            if minimo <= valor <= maximo:
                return valor
            print(f"⚠️ Por favor ingresa un número entre {minimo} y
{maximo}.")
        except ValueError:
            print("⚠️ Entrada no válida. Ingresa un número entero.")


def main():
    print("========================================")
    print("    REGISTRO DEL FESTIVAL DE MÚSICA     ")
    print("========================================")

    while True:
        try:
            num_artistas = int(input("¿Cuántos artistas se presentarán en el
festival?: "))
            if num_artistas > 0:
                break
            print("⚠️ Debes registrar al menos 1 artista.")
        except ValueError:
            print("⚠️ Por favor ingresa un número válido.")

    lista_artistas = []

    for i in range(1, num_artistas + 1):
        print(f"\n--- Datos del Artista #{i} ---")
        print("Selecciona el tipo de artista:")
        print("  1. Cantante")
        print("  2. DJ")
        print("  3. Banda")

        tipo = pedir_entero_rango("Opción (1-3): ", 1, 3)

        nombre = input("Nombre: ").strip()
        genero = input("Género musical: ").strip()
        popularidad = pedir_entero_rango("Popularidad (1 al 100): ", 1, 100)

        # Creación del objeto según el tipo seleccionado
        if tipo == 1:
            cancion = input("Canción más popular: ").strip()
            artista = Cantante(nombre, genero, popularidad, cancion)
        elif tipo == 2:
            estilo = input("Estilo de mezcla (ej. House, Techno, Electro):
").strip()
            artista = DJ(nombre, genero, popularidad, estilo)
        else:
            integrantes = pedir_entero_rango("Número de integrantes: ", 1,
100)
            artista = Banda(nombre, genero, popularidad, integrantes)

        lista_artistas.append(artista)

    # Iniciar festival utilizando polimorfismo
    iniciar_festival(lista_artistas)
```

```
if   name _ == "__main__":
    main()
```